

# Petcam — the score channel

The code that opens Discord on the laptop from the 5090: one PowerShell line on the mark, hands-free clicks and keystrokes from HQ.

Fabricio Ocampo · built & verified live 2026-10-04 (RowdyHacks XII)

HQ: 5090 (Omarchy, tailscale0 100.86.251.100) · Mark: Windows 11 laptop (100.80.61.26)

Repo: `~/apps/petcam` (git, commit `c75b549`)

## In this document

- 2 How the loop works — architecture & coordinate map
- 3 The one manual step: the PowerShell terminal on the laptop
- 4 — `drive.py` (HQ driver) — full annotated source
- 6 — `petcam-send-native.ps1` (laptop sender) — full annotated source

## How the loop works

Everything runs on plain HTTP over Tailscale. The laptop (the **mark**) runs one PowerShell script; the 5090 (HQ) runs the receiver, the brain, and the driver. Nothing is installed on the laptop beyond Windows itself.

```
LAPTOP (mark)                                5090 (HQ)
petcam-send-native.ps1                      receiver.py :9101
| frames -----> /frame POST → latest.jpg
| /hello (screen bounds + caps) -----> /client
| polls /control?after=N <----- /control (click/type/key events)
| applies them with SendInput on the laptop brain.py (watches feed health,
v drives the pet, consumes cmd.json)
the laptop's own mouse/keyboard drive.py (this doc) → writes cmd.json
```

## Coordinate map

Commands use **feed space (0-1280 x 0-720)**, which is the laptop's full virtual screen (1536x864 at origin 0,0) downscaled 16:9. The sender maps back exactly:  $px = ox + feedX * 1536 / 1280$ . The /hello handshake reports the real bounds, so HQ verifies the mapping instead of guessing.

## Verified recipe (opening Discord on the laptop)

```
python3 drive.py click 525 20                # aim: a blank Chrome tab (feed coords)
python3 drive.py open-url "https://discord.com" --no-focus
# open-url = ctrl+l (focus omnibox) → type url → enter → fresh screenshot
```

Rule of thumb: **re-shoot before every aim pass** — the mark's window layout changes under you. A journal verdict of no visible diff is NOT proof of failure; take a fresh drive.py shot and look at it.

## The one manual step: the PowerShell window on the laptop

The entire system needs exactly **one human action on the laptop**: open Windows PowerShell and run one line. Everything else (camera, click/type injection, the pet, this driver) runs itself. This page is that step, placed.

**1 Tailscale must be connected** on the laptop (same account as the 5090). Quick check: `tailscale status` — the 5090 shows up as `100.86.251.100`. No Tailscale = the `irm` below can't reach the receiver.

**2 Open Windows PowerShell** — any one of: Win+X then "Windows PowerShell" (or "Terminal"), Start menu search for "PowerShell", or Win+R → `powershell` → Enter. A regular (non-admin) window is fine — `SendInput` works for the current user session without elevation.

**3 Paste this one line and press Enter:**

```
iex (irm "http://100.86.251.100:9101/native.ps1")
```

That's it. `irm` (Invoke-RestMethod) downloads the script from HQ's receiver; (Invoke-Expression) runs it. The script downloads nothing else, installs nothing — it uses only built-in Windows/.NET pieces.

**4 Confirm the two banners** (appears in the same window, ~1s later):

```
petcam-native: streaming 1536x864 @ (0,0) → feed 1280x720 @ ~5fps to  
http://100.86.251.100:9101/frame  
petcam-native: hello → virtual 1536x864 @ (0,0), primary 1536x864
```

First line = camera is streaming. Second line = the handshake landed: HQ now knows the laptop's real screen bounds and can map clicks. From this moment both directions are open — frames flow to HQ, and HQ's clicks/types start landing on the laptop.

**5 Where to put the window: anywhere — ideally minimized.** The script is headless: it captures the *whole* virtual screen and injects input through `SendInput`, so the PowerShell window's own position doesn't matter. Minimize it (underscore button / Win+Down) and forget about it. **The only rule: don't close it.** Closing the window stops the camera *and* the score channel together (one process does both).

### Keeping it alive — the one historical bug

**Re-running in the same window used to kill the stream.** Running the line a second time in the *same* PowerShell session threw `Add-Type : Cannot add type. The type name 'PetcamInput' already exists and aborted the script` (the compiled type survives per-session). The script now guards the `Add-Type` with `if (-not ("PetcamInput" -as [type]))`, so re-runs in the same window work. Cleanest is still a **fresh PowerShell window** for a re-run. Symptom of a dead stream: HQ sees stale /state (frame age > 10s) and the laptop window shows the red `Add-Type` error — open a new window, re-run the line, done in ~5 seconds.

## drive.py — HQ driver (the working buttons)

Lives at `~/apps/petcam/drive.py` on the 5090. CLI entry into the score channel: it writes `cmd.json`, the brain applies the event on the laptop, and the driver waits for the journal verdict (outcome + screen diff) before reporting. Python 3, stdlib only.

```
#!/usr/bin/env python3
"""petcam drive: the proven score-channel driver for the mark's machine.

Saved 2026-10-04 after live verification: clicks and unicode types land on
the mark (feed 1280x720 = the mark's full virtual screen, downscaled; the
native.ps1 sender maps back with px = ox + feedX*screenW/1280).

Usage (all coords in FEED space, 0-1280 x 0-720):
drive.py shot [path]          # fetch /latest.jpg (default inbox/shot-<ts>.jpg)
drive.py click X Y [btn]     # left-click at feed X,Y (btn 2 = right)
drive.py type "text"         # unicode type at the current cursor
drive.py key KEY [MOD ...]   # enter, l, esc, ... mods: ctrl alt shift win
drive.py open-url "URL"      # chrome recipe: focus omnibox (ctrl+l), type, enter
drive.py state               # feed health one-liner

Every action waits on the journal (brain writes the outcome + screen diff)
and reports it. Pitfalls baked in:
- changed:false / diff 0.0 does NOT mean the input failed: the verify
  frame is captured milliseconds after the event, and Chrome tab/window
  transitions can paint a frame later. Verify with a FRESH shot + vision.
- The mark's window layout changes under you (the user rearranges things).
  Re-shoot before every aim pass; don't reuse stale coords across passes.
- ctrl+l is the robust omnibox focus (no aiming) once the Chrome window
  itself has focus. Give Chrome focus by clicking inside it first.
"""
import json
import os
import sys
import time
import urllib.request

HERE = os.path.dirname(os.path.abspath(__file__))
BASE = os.environ.get("PETCAM_BASE", "http://100.86.251.100:9101")
CMD_FILE = os.path.join(HERE, "cmd.json")
JOURNAL = os.path.join(HERE, "journal.jsonl")

def get(path, timeout=5):
    with urllib.request.urlopen(BASE + path, timeout=timeout) as r:
        return r.read()

def shot(path=None):
    if path is None:
        os.makedirs(os.path.join(HERE, "inbox"), exist_ok=True)
        path = os.path.join(HERE, "inbox", "shot-" + time.strftime("%H%M%S") + ".jpg")
    open(path, "wb").write(get("/latest.jpg"))
    return path

def state():
    st = json.loads(get("/state"))
    age = time.time() - st["last_frame_at"]
    ctl = time.time() - st.get("last_control_poll_at", 0)
    health = "live" if age < 10 else "stale" if age < 120 else "dead"
    print(f"{health}: frame age {age:.1f}s, ctrl poll {ctl:.1f}s, "
          f"frames {st['frames']}, client {st['last_client']}")
    return health

def _journal_mark():
    try:
        return os.path.getsize(JOURNAL)
    except OSError:
        return 0

def _journal_since(mark):
    with open(JOURNAL, "rb") as f:
        f.seek(mark)
        data = f.read().decode("utf-8", "replace")
    out = []
    for ln in data.strip().splitlines():
        try:
```

```

        out.append(json.loads(ln))
    except Exception:
        pass
    return out

def send(cmd, wait=True, timeout=20):
    """Write cmd.json; the brain consumes it and journals the outcome."""
    with open(CMD_FILE, "w") as f:
        json.dump(cmd, f)
    if not wait:
        time.sleep(1.0)
        return None
    mark = _journal_mark()
    deadline = time.time() + timeout
    while time.time() < deadline:
        for e in _journal_since(mark):
            if e.get("cmd") == cmd.get("cmd"):
                time.sleep(0.4) # let the brain consume the cmd file
                return e
        time.sleep(0.8)
    return None

def _verdict(e):
    if e is None:
        return "no journal entry (brain down?)"
    ch = e.get("changed")
    return {True: "mark responded (diff %s)" % e.get("diff"),
            False: "no visible diff (verify with a fresh shot - not proof of failure)",
            None: "unverifiable"}[ch]

def main():
    a = sys.argv[1:]
    if not a:
        print(__doc__)
        return 1
    c = a[0]
    if c == "shot":
        print(shot(a[1] if len(a) > 1 else None))
    elif c == "state":
        state()
    elif c == "click":
        x, y = float(a[1]), float(a[2])
        btn = int(a[3]) if len(a) > 3 else 1
        e = send({"cmd": "click", "x": x, "y": y, "button": btn})
        print(f"click {x},{y} btn{btn}: {e.get('result')} if e else '?'; {_verdict(e)}")
    elif c == "type":
        e = send({"cmd": "type", "text": a[1]})
        print(f"type {len(a[1])} chars: {e.get('result')} if e else '?'; {_verdict(e)}")
    elif c == "key":
        key = a[1].lower()
        mods = [m.lower() for m in a[2:]]
        e = send({"cmd": "key", "key": key, "mods": mods})
        print(f"key {key} {'+'.join(mods) if mods else ''}: {e.get('result')} if e else '?'; {_verdict(e)}")
    elif c == "open-url":
        url = a[1]
        if len(a) > 2 and a[2] != "--no-focus":
            # give Chrome window focus first: caller must pass the feed coords
            # of a safe point inside the Chrome window
            fx, fy = float(a[3]), float(a[4])
            e = send({"cmd": "click", "x": fx, "y": fy, "button": 1})
            print(f"focus chrome at {fx},{fy}: {e.get('result')} if e else '?'; {_verdict(e)}")
            time.sleep(0.8)
            e = send({"cmd": "key", "key": "l", "mods": ["ctrl"]})
            print(f"ctrl+l: {e.get('result')} if e else '?'; {_verdict(e)}")
            time.sleep(0.8)
            e = send({"cmd": "type", "text": url})
            print(f"type {url}: {e.get('result')} if e else '?'; {_verdict(e)}")
            time.sleep(0.8)
            e = send({"cmd": "key", "key": "enter"})
            print(f"enter: {e.get('result')} if e else '?'; {_verdict(e)}")
            time.sleep(3)
            print(shot())
        else:
            print(__doc__)
            return 1
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

## petcam-send-native.ps1 — the zero-install sender (laptop side)

Served by the receiver at /native.ps1; the laptop fetches it on demand via iex (irm ...). One process does both jobs: JPEG screen capture at ~5fps to /frame, and polling /control to apply HQ's click/type/key events with SendInput. No ffmpeg, no Python, nothing installed — built-in System.Drawing + WebClient + user32 P/Invoke only.

```
# petcam-send-native.ps1 - zero-install screen streamer + score channel for Windows 10/11.
# No ffmpeg, no Python, nothing installed: built-in .NET System.Drawing (CopyFromScreen)
# + built-in WebClient to POST JPEG frames, + user32 P/Invoke to apply HQ's click/type
# events on the mark's own machine. Close this window to stop.
# Run (Tailscale connected): iex (irm "http://100.86.251.100:9101/native.ps1")
#
# WHY THIS EXISTS (the "clean getaway" camera)
# The primary camera for the mark: one line on the laptop, iex (irm ...).
# Everything it uses ships with Windows, so the hackathon demo needs zero
# installs on the remote machine (the user's explicit constraint).
#
# COORDINATE MAP (why clicks land where the feed shows)
# The feed captures the FULL VIRTUAL SCREEN (all monitors, origin may be
# negative) downsampled to a FIXED 1280x720. HQ pushes events in feed space;
# this script maps them back to absolute virtual-screen pixels:
# px = VS.X + feedX * VS.Width / 1280
# py = VS.Y + feedY * VS.Height / 720
# /hello (POSTed at connect) reports the real bounds + capabilities to HQ
# so HQ can verify the mapping instead of guessing.
#
# WHY THE INPUT BLOCK IS P/Invoke (SendInput + SetCursorPos)
# The SCORE channel: the camera is the only thing on the mark's machine, so
# it is also the only thing that can inject input there. Layout rules that
# matter (the old version got these wrong and clicks/types silently no-op'd):
# - INPUTUNION must be Explicit with BOTH fields at FieldOffset(0) - a
#   Sequential union puts ki at offset 24, so keyboard events wrote past
#   the union and SendInput read garbage (wVk=0).
# - INPUT: uint type at 0, union at offset 8 (pointer-aligned). Natural
#   alignment via default StructLayout; SizeOf computed at call time.
# - Button state goes in dwFlags (LEFTDOWN=2/LEFTUP=4/RIGHTDOWN=8/RIGHTUP=10),
#   NOT in mouseData (mouseData is wheel-only).
# - Absolute positioning via SetCursorPos (virtual-screen coords, can go
#   negative / off-screen to other monitors) - SendInput relative deltas
#   accumulate drift and can't target an absolute point.
# - Unicode typing: KEYEVENTF_UNICODE (0x0004) + wScan=charcode, wVk=0.
#   Plain wVk=char only works for A-Z/0-9, so lowercase and '.' were lost.
# - SendInput returns the count injected; 0 = failure → GetLastError.
# - WebClient.Timeout setter throws on .NET Framework 4.x - removed.
# - [void] on UploadData so the "ok" response body doesn't print per frame.
$ErrorActionPreference = "Continue"
$target = if ($env:PETCAM_TARGET) { $env:PETCAM_TARGET } else { "http://100.86.251.100:9101/frame" }
$w = 1280; $h = 720 # FIXED feed - HUD, window sizes and brain aim
# math are built around 16:9 1280x720.
$fps = if ($env:PETCAM_FPS) { [int]$env:PETCAM_FPS } else { 5 }

Add-Type -AssemblyName System.Drawing
Add-Type -AssemblyName System.Windows.Forms

# ---- Score channel: apply click/type events from HQ (SendInput + SetCursorPos) ----
# Idempotent: re-running iex (irm ...) in the SAME session used to throw
# TYPE_ALREADY_EXISTS at Add-Type and kill the script (the type survives per
# session). Skip the Add-Type if PetcamInput is already registered.
if (-not ("PetcamInput" -as [type])) {
    Add-Type @"
using System;
using System.Runtime.InteropServices;
public static class PetcamInput {
    [StructLayout(LayoutKind.Sequential)]
    public struct MOUSEINPUT {
        public int dx, dy, mouseData, dwFlags, time;
        public IntPtr dwExtraInfo;
    }
    [StructLayout(LayoutKind.Sequential)]
    public struct KEYBDINPUT {
        public ushort wVk, wScan;
        public uint dwFlags, time;
        public IntPtr dwExtraInfo;
    }
    [StructLayout(LayoutKind.Explicit)]
    public struct INPUTUNION {
        [FieldOffset(0)] public MOUSEINPUT mi;
        [FieldOffset(0)] public KEYBDINPUT ki;
    }
    [StructLayout(LayoutKind.Sequential)]
    public struct INPUT {
        public uint type;
        public INPUTUNION u; // natural alignment: union lands at offset 8 on 64-bit
    }
    [DllImport("user32.dll")] static extern uint SendInput(uint n, INPUT[] p, int cb);
    [DllImport("user32.dll")] static extern bool SetCursorPos(int x, int y);
    [DllImport("kernel32.dll")] static extern uint GetLastError();
    const uint MOUSEEVENTF_LEFTDOWN=0x0002, MOUSEEVENTF_LEFTUP=0x0004,
        MOUSEEVENTF_RIGHTDOWN=0x0008, MOUSEEVENTF_RIGHTUP=0x0010;
    const ushort KEYEVENTF_KEYUP=0x0002, KEYEVENTF_UNICODE=0x0004;
    static uint Check(uint n, uint injected) {
        if (injected != n) throw new System.ComponentModel.Win32Exception((int)GetLastError(), "SendInput injected " + injected + "/" + n);
        return injected;
    }
    static INPUT Mouse(uint flags) {
        INPUT i = new INPUT(); i.type = 0; i.u.mi.dwFlags = (int)flags; return i;
    }
    static INPUT Key(ushort vk, ushort flags) {
        INPUT i = new INPUT(); i.type = 1; i.u.ki.wVk = vk; i.u.ki.dwFlags = flags; return i;
    }
    public static void Move(int x, int y) {
        if (!SetCursorPos(x, y)) throw new System.ComponentModel.Win32Exception((int)GetLastError(), "SetCursorPos(" + x + ", " + y + ")");
    }
}
```

```

public static void Click(int x, int y, bool right) {
    Move(x, y);
    uint down = right ? MOUSEEVENTF_RIGHTDOWN : MOUSEEVENTF_LEFTDOWN;
    uint up = right ? MOUSEEVENTF_RIGHTUP : MOUSEEVENTF_LEFTUP;
    INPUT[] d = { Mouse(down) }, u = { Mouse(up) };
    Check(1, SendInput(1, d, Marshal.SizeOf(typeof(INPUT))));
    Check(1, SendInput(1, u, Marshal.SizeOf(typeof(INPUT))));
}

public static void Type(string s) {
    int n = s.Length * 2;
    INPUT[] k = new INPUT[n];
    for (int i = 0; i < s.Length; i++) {
        k[i*2] = new INPUT() { type=1, u=new INPUTUNION() { ki=new KEYBDINPUT() { wScan=(ushort)s[i], dwFlags=KEYEVENTF_UNICODE } } };
        k[i*2+1] = new INPUT() { type=1, u=new INPUTUNION() { ki=new KEYBDINPUT() { wScan=(ushort)s[i], dwFlags=(ushort)(KEYEVENTF_UNICODE|KEYEVENTF_KEYUP) } } };
    }
    Check((uint)n, SendInput((uint)n, k, Marshal.SizeOf(typeof(INPUT))));
}

public static void Key(ushort vk, string[] mods) {
    ushort[] mkeys = new ushort[4]; int mn = 0;
    if (mods != null) foreach (string m in mods) {
        switch (m) { case "ctrl": mkeys[mn++] = 0x11; break; case "alt": mkeys[mn++] = 0x12; break;
                    case "shift": mkeys[mn++] = 0x10; break; case "win": mkeys[mn++] = 0x5B; break; }
    }
    int n = mn*2 + 2;
    INPUT[] k = new INPUT[n]; int p = 0;
    for (int i = 0; i < mn; i++) { k[p++] = Key(mkeys[i], 0); }
    k[p++] = Key(vk, 0);
    k[p++] = Key(vk, KEYEVENTF_KEYUP);
    for (int i = mn-1; i >= 0; i--) { k[p++] = Key(mkeys[i], KEYEVENTF_KEYUP); }
    Check((uint)n, SendInput((uint)n, k, Marshal.SizeOf(typeof(INPUT))));
}

}
"@
}

$ctrlBase = ($target -replace '/frame$', '')
$script:lastSeq = 0
function Poll-Control {
    param($Web)
    try {
        $r = $Web.DownloadString("$ctrlBase/control?after=$script:lastSeq")
        $j = $r | ConvertFrom-Json
        # Receiver restart => its seq counter reset below our local cursor.
        # Detect the regression, rewind to 0, and re-fetch so we don't skip events.
        if ($j -and $j.seq -lt $script:lastSeq) {
            $script:lastSeq = 0
            $r = $Web.DownloadString("$ctrlBase/control?after=0")
            $j = $r | ConvertFrom-Json
            Write-Host ("score: control seq reset detected, re-synced at seq {0}" -f $j.seq)
        }
        if ($null -ne $j) {
            foreach ($e in $j.events) {
                if ($null -eq $e) { continue }
                try {
                    if ($e.type -eq 'click') {
                        $px = [int]([Math]::Round($screen0x + $e.x * $screenW / $ow))
                        $py = [int]([Math]::Round($screen0y + $e.y * $screenH / $oh))
                        [void][PetcamInput]::Click($px, $py, ($e.button -eq 2))
                        Write-Host ("score: click feed {0},{1} -> screen {2},{3} (btn {4})" -f $e.x, $e.y, $px, $py, $e.button)
                    }
                    elseif ($e.type -eq 'move') {
                        $px = [int]([Math]::Round($screen0x + $e.x * $screenW / $ow))
                        $py = [int]([Math]::Round($screen0y + $e.y * $screenH / $oh))
                        [void][PetcamInput]::Move($px, $py)
                        Write-Host ("score: move feed {0},{1} -> screen {2},{3}" -f $e.x, $e.y, $px, $py)
                    }
                    elseif ($e.type -eq 'type') {
                        [void][PetcamInput]::Type($e.text)
                        Write-Host ("score: typed {0} chars" -f $e.text.Length)
                    }
                    elseif ($e.type -eq 'key') {
                        # Named key + optional modifiers: {"key":"t","mods":["ctrl"]}
                        # mods: ctrl, alt, shift, win
                        $vk = $null
                        switch ($e.key) {
                            'enter' { $vk = 0x0D }; 'tab' { $vk = 0x09 }; 'esc' { $vk = 0x1B }
                            'space' { $vk = 0x20 }; 'backspace' { $vk = 0x08 }; 'delete' { $vk = 0x2E }
                            'insert' { $vk = 0x2D }; 'home' { $vk = 0x24 }; 'end' { $vk = 0x23 }
                            'pageup' { $vk = 0x21 }; 'pagedown' { $vk = 0x22 }
                            'up' { $vk = 0x26 }; 'down' { $vk = 0x28 }; 'left' { $vk = 0x25 }; 'right' { $vk = 0x27 }
                            'f1' { $vk = 0x70 }; 'f2' { $vk = 0x71 }; 'f3' { $vk = 0x72 }; 'f4' { $vk = 0x73 }
                            'f5' { $vk = 0x74 }; 'f6' { $vk = 0x75 }; 'f7' { $vk = 0x76 }; 'f8' { $vk = 0x77 }
                            'f9' { $vk = 0x78 }; 'f10' { $vk = 0x79 }; 'f11' { $vk = 0x7A }; 'f12' { $vk = 0x7B }
                            default {
                                if ($e.key -and $e.key.Length -eq 1) { $vk = [int][char]($e.key.ToUpperInvariant()) }
                            }
                        }
                    }
                    if ($null -eq $vk) {
                        Write-Host ("score: key ? unknown key '{0}'" -f $e.key)
                    }
                    else {
                        [void][PetcamInput]::Key($vk, @($e.mods))
                        Write-Host ("score: key '{0}' mods {1}" -f $e.key, ($e.mods -join ','))
                    }
                }
            }
        } catch {
            Write-Host ("score: apply failed: {0}" -f $_.Exception.Message)
        }
        if ($e.seq -gt $script:lastSeq) { $script:lastSeq = $e.seq }
    }
} catch {
    # control poll is best-effort; frames keep flowing regardless
}

# JPEG encoder, quality 75 (built-in codec)
$enc = [System.Drawing.Imaging.ImageCodecInfo]::GetImageEncoders() |
    Where-Object { $_.MimeType -eq 'image/jpeg' } | Select-Object -First 1
$ep = New-Object System.Drawing.Imaging.EncoderParameters(1)
$ep.Param[0] = New-Object System.Drawing.Imaging.EncoderParameter([System.Drawing.Imaging.Encoder]::Quality, [long]75)

$web = New-Object System.Net.WebClient
$web.Headers.Add("Content-Type", "image/jpeg")
# NOTE: no .Timeout assignment - WebClient.Timeout setter throws on .NET 4.x.

```

```

# FULL VIRTUAL SCREEN (all monitors; origin can be negative) is the capture +
# click-mapping space, so the feed and the score channel cover every display.
$vsb = [System.Windows.Forms.SystemInformation]::VirtualScreen
$screenOx = $vsb.X; $screenOy = $vsb.Y; $screenW = $vsb.Width; $screenH = $vsb.Height
Write-Host "petcam-native: streaming ${screenW}x${screenH} @ (${screenOx},${screenOy}) → feed ${ow}x${oh} @ ~${fps}fps to $target"
Write-Host "close this window to stop"

# Handshake: report real screen bounds + capabilities so HQ can verify the
# feed→screen mapping instead of guessing.
try {
    $vs = [System.Windows.Forms.SystemInformation]::VirtualScreen
    $ps = [System.Windows.Forms.Screen]::PrimaryScreen.Bounds
    $payload = @{
        host = $env:COMPUTERNAME
        os = [System.Environment]::OSVersion.VersionString
        screen = @{ w = $vs.Width; h = $vs.Height; ox = $vs.X; oy = $vs.Y }
        primary = @{ x = $ps.X; y = $ps.Y; w = $ps.Width; h = $ps.Height }
        dpi = [System.Windows.Forms.SystemInformation]::DeviceDPI
        caps = @("click", "move", "type", "unicode", "key", "cursor")
        feed = @{ w = $ow; h = $oh; fps = $fps }
    }
    $hw = New-Object System.Net.WebClient
    [void]$hw.UploadString("$ctrlBase/hello", "POST", (ConvertTo-Json $payload -Depth 5))
    Write-Host ("petcam-native: hello → virtual {0}x{1} @ ({2},{3}), primary {4}x{5}" -f
        $vs.Width, $vs.Height, $vs.X, $vs.Y, $ps.Width, $ps.Height)
} catch {
    Write-Host ("petcam-native: hello failed (non-fatal): {0}" -f $_.Exception.Message)
}

$i = 0
while ($true) {
    $t0 = [DateTime]::Now
    $full = $null; $shrink = $null; $ms = $null
    Poll-Control $web
    try {
        $full = New-Object System.Drawing.Bitmap($screenW, $screenH)
        $g = [System.Drawing.Graphics]::FromImage($full)
        $g.CopyFromScreen($screenOx, $screenOy, 0, 0, $full.Size, [System.Drawing.CopyPixelOperation]::SourceCopy)
        $g.Dispose()
        $shrink = New-Object System.Drawing.Bitmap($ow, $oh, [System.Drawing.Imaging.PixelFormat]::Format32bppArgb)
        $gs = [System.Drawing.Graphics]::FromImage($shrink)
        $gs.InterpolationMode = [System.Drawing.Drawing2D.InterpolationMode]::Bilinear
        $gs.DrawImage($full, 0, 0, $ow, $oh)
        $gs.Dispose()
        $full.Dispose(); $full = $null
        $ms = New-Object System.IO.MemoryStream
        $shrink.Save($ms, $enc, $sep)
        $shrink.Dispose(); $shrink = $null
        $bytes = $ms.ToArray()
        [void]$web.UploadData($target, $bytes)
        $i++
    } catch {
        Write-Host ("petcam-native: frame error: {0}" -f $_.Exception.Message)
    } finally {
        if ($ms) { $ms.Dispose() }
        if ($full) { $full.Dispose() }
        if ($shrink) { $shrink.Dispose() }
    }
    $elapsed = ([DateTime]::Now - $t0).TotalMilliseconds
    $sleepMs = [Math]::Max(150, [int](1000.0 / $fps - $elapsed))
    Start-Sleep -Milliseconds $sleepMs
}

```